1. Digital Class Notes App

Selected Data Structure Method: *Stack and Hash Table*
Reason: In this app, the user frequently edits notes and may make mistakes. To support the undo function, a **stack** is the best choice because it follows the Last-In-First-Out (LIFO) method — it removes the most recent action first, which is exactly how undo works. For organizing and finding notes by subject or title (like "Linear Search"), a **hash table** is useful because it allows for very fast search, insert, and retrieve operations using keywords as keys.

In a digital class notes app, subjects like Data Structure, Computer Networks, and Operating Systems would be included. Topics may involve Linked Lists, Queues, Trees, etc. If a mistake is made while editing, the undo feature helps reverse the last change instantly. To find a note like “Linear Search,” a search bar can be used which matches keywords and quickly locates the note among all saved ones.

2. Fast-Food Order Management

Selected Data Structure Method: *Queue and Priority Queue*
Reason: In a fast-food restaurant, customers are usually served in the order they arrive, which matches the **queue** structure (First-In-First-Out). However, VIP customers must be served before others even if they come later. This requires a **priority queue**, which allows certain elements (VIP orders) to be served ahead of others based on their priority. To manage order cancellations, searching by unique order ID can be done using additional indexing or a hash map.

In this scenario, assume 3 customers: Alice orders Burger, Bob orders Fries, and Carol orders Pizza. If a VIP like Carol arrives late, a priority queue system ensures she is served before others. If a customer cancels after 5 minutes, the system searches for the order by ID and deletes it from the queue, maintaining an updated order list.

3. Library Book Tracker

Selected Data Structure Method: *Stack and Set*
Reason: Recently returned books should be shown first, which makes **stack** a suitable data structure — it follows the LIFO method, so the most recently returned book is accessed first. To prevent duplicate book entries (for example, if someone tries to return a book twice), a **set** is useful. Sets automatically avoid duplication and are efficient for checking whether a book is already listed.

Books I would borrow from the department library are 'Introduction to Algorithms', 'C Programming Language', and 'Computer Networks'. Recently returned books are stored in a

stack to maintain recent order. For the end-of-day sorting, a sorting algorithm like Merge Sort can organize the list alphabetically. Duplicates are avoided using a set or by checking before insertion.

4. Maze Puzzle App

Selected Data Structure Method: *Stack and Set*
Reason: In a maze game, players may go down a path that leads to a dead-end. When this happens, **stack** helps backtrack — removing the last move and trying a new one. This works because stack keeps the history of previous moves in the correct order. To avoid retrying already failed paths, a **set** can be used to store visited locations or paths and check before retrying.

I would design a maze called 'Zombie Land' with 6 paths and 10 turns. If a player reaches a dead-end, the game uses backtracking to return to the last decision point and try a new path. Already tried paths are stored using a set or list to avoid repeating failed routes.

5. Smart Daily Planner

Selected Data Structure Method: *Priority Queue*
Reason: In a daily planner app, tasks are not always equal — some are urgent and need to be done first. A **priority queue** helps to manage tasks by their urgency levels. Urgent tasks are automatically placed higher in the queue, and less important ones are scheduled after them. This structure helps efficiently choose the next task based on priority and also supports rescheduling by changing the priority or removing a task from the queue.

Tasks I did today include 'Study Data Structure', 'Attend class', and 'Buy groceries'. These are scheduled based on urgency. If a teacher gives a surprise task, it is marked as urgent and pushed to the top of the list using a priority queue. The next task is chosen based on priority and time availability.

6. Bus Terminal Management

Selected Data Structure Method: *Queue and Array*
Reason: Buses depart in order, and a **queue** is the best fit to log their departure — buses that arrive first are logged and leave first. Additionally, the list of all buses by route number should be displayed in sorted order. For this, an **array** is ideal as it allows sorting using algorithms like Merge Sort or Quick Sort. Arrays are also easy to traverse and update when a bus leaves early.

Bus names in my area include GreenLine, Shyamoli, and Hanif. If a bus leaves early, its log is updated immediately using a queue system. For alphabetical sorting of buses, a sorting

algorithm like Quick Sort is used. This ensures buses are organized and accessible for report generation.

7. Music Playlist Manager

Selected Data Structure Method: *Queue and Circular Queue*
Reason: Songs play one by one in order, so a **queue** structure is suitable. Once the last song ends, the playlist starts again from the beginning — this continuous looping behavior is supported by a **circular queue**. If a user wants to go back and replay a recent song, a temporary stack or double-ended queue (deque) may also be used to manage playback history.

My playlist includes: 'Believer', 'Shape of You', 'On My Way', 'Faded', 'Senorita'. Songs play in sequence using a queue. To repeat a song, the player navigates backward using a circular pointer or stores recent songs in a stack. When the last song ends, it loops back to the first using circular queue logic.

8. College Event Registration

Selected Data Structure Method: *Array and Sorting Algorithm*
Reason: Registrations can be stored in an **array**, which is efficient for inserting and tracking the list of participants. If someone cancels, their name can be removed by searching the array. At the end, the list must be displayed alphabetically, which can be done by applying a **sorting algorithm** such as Bubble Sort or Quick Sort. Arrays make it easy to implement and display sorted data.

Assume the event is 'Tech Carnival'. Three registered students: Rafi, Sumi, and Zayed. If Sumi cancels, her name is removed using a linear search. The remaining list is then sorted alphabetically using bubble sort before final display.

9. Exam Script Checker

Selected Data Structure Method: *Array and Max Heap*
Reason: Student roll numbers and scores are stored in an **array**, which allows direct indexing and updating scores. To continuously track the top 5 scorers, a **max heap** is useful — it keeps the highest values at the top and updates automatically when scores change (like after rechecking). This method allows you to efficiently retrieve or display the highest scores without needing to sort the full array each time.

Assume roll numbers: 101 (85), 102 (92), 103 (78), 104 (88). If roll 103 requests a recheck and scores 90, the top 5 scores list is updated by replacing the old score. Scores are sorted in descending order using a simple sort to show rankings.

10.Hospital Emergency Room

Selected Data Structure Method: *Priority Queue and Queue*

Reason: In a hospital emergency room, critical patients must be treated before regular ones, regardless of arrival time. A **priority queue** handles this by assigning higher priority to critical cases. Normal patients are managed using a regular **queue**, which logs arrivals in order. This combination ensures that life-threatening cases are handled urgently, while others are treated in their turn.

Patients: Rahim (normal fever), Karim (critical injury), and Tania (flu). Critical patient Karim is treated first due to priority queue handling. If a new critical case like Jamal arrives, he is inserted at the front and treated before other non-critical cases.

CT questions

CT Set 1: Circular Queue and Fibonacci Function Calls

1. In a bus terminal ticketing system, passengers are served using a circular queue with size n = 5. The system processes the following operations: Enqueue A, B, C → Dequeue twice → Enqueue D, E, F. Simulate the final state. Write the final queue content, positions of front and rear, and the condition that checks if the queue is full.

In a circular queue of size 5, we perform the following operations:

- Enqueue A, B, C → Queue becomes: A, B, C
- Dequeue twice → A and B are removed → Queue: C
- Enqueue D, E, F → Now queue: C, D, E, F

Step-by-step Simulation:

Step	Action	Queue Status	Front	Rear
Start	Empty	[- , - , - , - , -]	-	-
Enqueue A	A	[A , - , - , - , -]	0	0
Enqueue B	B	[A , B , - , - , -]	0	1
Enqueue C	C	[A , B , C , - , -]	0	2
Dequeue	removes A	[- , B , C , - , -]	1	2
Dequeue	removes B	[- , - , C , - , -]	2	2
Enqueue D	D	[- , - , C , D , -]	2	3
Enqueue E	E	[- , - , C , D , E]	2	4
Enqueue F	F	[F , - , C , D , E]	2	0

Front starts at A (0), after two dequeues it's at C (index 2). Rear ends at F (index 0 due to circular wrap-around).

To check if the queue is full: (rear + 1) % size == front → (0+1)%5 == 2 → Not full.

Final Queue: [F, _, C, D, E] (with front at index 2 and rear at index 0).

2. In a research lab, a species' population grows following the Fibonacci pattern: F(n) = F(n-1) + F(n-2) with F(0) = 0 and F(1) = 1. For a recursive approach to calculate F(5), determine the 5th Fibonacci number and count how many times the function is called in total.

Fibonacci Rule:

F(n) = F(n-1) + F(n-2)

Base:

F(0) = 0, F(1) = 1

```
F(5)
├──── F(4)
│    ├──── F(3)
│    │    ├──── F(2)
│    │    │    ├──── F(1) → 1
│    │    │    └──── F(0) → 0
│    │    └──── F(1) → 1
│    └──── F(2)
│         ├──── F(1) → 1
│         └──── F(0) → 0
└──── F(3)
     ├──── F(2)
     │    ├──── F(1) → 1
     │    └──── F(0) → 0
     └──── F(1) → 1
```

For Fibonacci using recursion:
F(5) = F(4) + F(3)
F(4) = F(3) + F(2)
F(3) = F(2) + F(1)
F(2) = F(1) + F(0)
F(1) = 1
F(0) = 0
...
Total function calls: 15
F(5) = 5

CT Set 2: Circular Queue with Issues and Fibonacci Sum

1.In a bus terminal ticketing system, a circular queue of size n = 5 is used to manage passengers. The system performs the following operations: Dequeue once → Enqueue A, B, C → Dequeue twice → Enqueue D, E, F. Simulate the final state of the queue. Write the final queue content, the positions of front and rear, and mention if any problem occurs during these operations. If a problem occurs, explain why and how it can be solved.

Operations:
- Dequeue once (initially empty): Error! Queue underflow – no data to remove.
To fix this, ensure the queue is not empty before dequeuing.

Initial Setup

- Circular Queue Size = 5
- Operations:
    1. Dequeue once
    2. Enqueue A
    3. Enqueue B
    4. Enqueue C
    5. Dequeue twice
    6. Enqueue D
    7. Enqueue E
    8. Enqueue F

Step	Action	Queue Status	Front	Rear
Start	Empty	[- , - , - , - , -]	-1	-1
Dequeue	Error: Underflow	[- , - , - , - , -]	-1	-1
Enqueue A	A	[A , - , - , - , -]	0	0
Enqueue B	B	[A , B , - , - , -]	0	1
Enqueue C	C	[A , B , C , - , -]	0	2
Dequeue	removes A	[- , B , C , - , -]	1	2
Dequeue	removes B	[- , - , C , - , -]	2	2
Enqueue D	D	[- , - , C , D , -]	2	3
Enqueue E	E	[- , - , C , D , E]	2	4
Enqueue F	F	[F , - , C , D , E]	2	0

Problem: Dequeue at start causes error. Solution: validate before operation.

2.In a research lab, a species' population grows following the Fibonacci pattern: F(n) = F(n-1) + F(n-2), where F(0) = 0 and F(1) = 1. Write a recursive function to calculate F(5). Then, using this recursive approach, compute the sum of the Fibonacci numbers from F(1) to F(5).

Fibonacci Recursive Function in C:

```c
int fib(int n) {
   if (n <= 1) return n;
   return fib(n-1) + fib(n-2);
}
```

```
F(5)
├─── F(4)
│   ├─── F(3)
│   │   ├─── F(2)
│   │   │   ├─── F(1) = 1
│   │   │   └─── F(0) = 0
│   │   └─── F(1) = 1
│   └─── F(2)
│       ├─── F(1) = 1
│       └─── F(0) = 0
└─── F(3)
    ├─── F(2)
    │   ├─── F(1) = 1
    │   └─── F(0) = 0
    └─── F(1) = 1
```

F(1)=1, F(2)=1, F(3)=2, F(4)=3, F(5)=5 → Sum = 1+1+2+3+5 = 12

CT Set A: Singly Linked List and Node Insertion

A singly linked list is a structure where each node points to the next, forming a chain. Unlike arrays, linked lists don't require contiguous memory and can grow dynamically.

Insertion example in C:

```c
struct Node* newNode = malloc(sizeof(struct Node));
newNode->data = 30;
newNode->next = q->next;
q->next = newNode;
```

Assuming q points to node with value 25, the new node with 30 is inserted before 15.

CT Set B: Linked List Advantages and Deletion

Advantages of Linked List:

1. Dynamic memory: It grows/shrinks as needed.
2. Easy insertion/deletion: O(1) time if pointer is known.

Deletion in C:

```c
struct Node* temp = p->next;
p->next = temp->next;
free(temp);
```

This removes the node after value 12, assuming p points to 12.